

Chapter 12

Integration Testing



The Mars Climate Orbiter Mission

- mission failed in September 1999
- completed successful flight: 416,000,000 miles (665.600.600 km)
- 41 weeks flight duration
- lost at beginning of Mars orbit
- An integration fault: Lockheed Martin used English units for acceleration calculations (pounds), and Jet Propulsion Laboratory used metric units (newtons).
- NASA announced a US\$ 50,000 project to discover how this happened. (We know!)
- They got it right in 2021



Goals/Purpose of Integration Testing

- Presumes previously tested units
- Not system testing
- Tests functionality "between" unit and system levels
- Basis for test case identification?
- Emphasis shifts from “how to test” to “what to test” (Model-Based Testing)



Testing Level Assumptions and Objectives

- Unit assumptions
 - All other units are correct
 - Compiles correctly
- Integration assumptions
 - Unit testing complete
- System assumptions
 - Integration testing complete
 - Tests occur at port boundary
- Unit goals
 - Correct unit function
 - Coverage metrics satisfied
- Integration goals
 - Interfaces correct
 - Correct function across units
 - Fault isolation support
- System goals
 - Correct system functions
 - Non-functional requirements tested
 - Customer satisfaction.



Approaches to Integration Testing

(“source” of test cases)

- Functional Decomposition (most commonly described in the literature)
 - Top-down
 - Bottom-up
 - Sandwich
 - “Big bang”
- Call graph
 - Pairwise integration
 - Neighborhood integration
- Paths
 - MM-Paths
 - Atomic System Functions



Basis of Integration Testing Strategies

- Functional Decomposition
applies best to procedural code
- Call Graph
applies to both procedural and object-oriented code
- MM-Paths
applies to both procedural and object-oriented code



Continuing Example—Calendar Program

- (see text for pseudo-code version)
- Date in the form mm, dd, yyyy
- Calendar functions
 - the date of the next day (our old friend, NextDate)
 - the day of the week corresponding to the date
 - the zodiac sign of the date
 - the most recent year in which Memorial Day was celebrated on May 27
 - the most recent Friday the Thirteenth

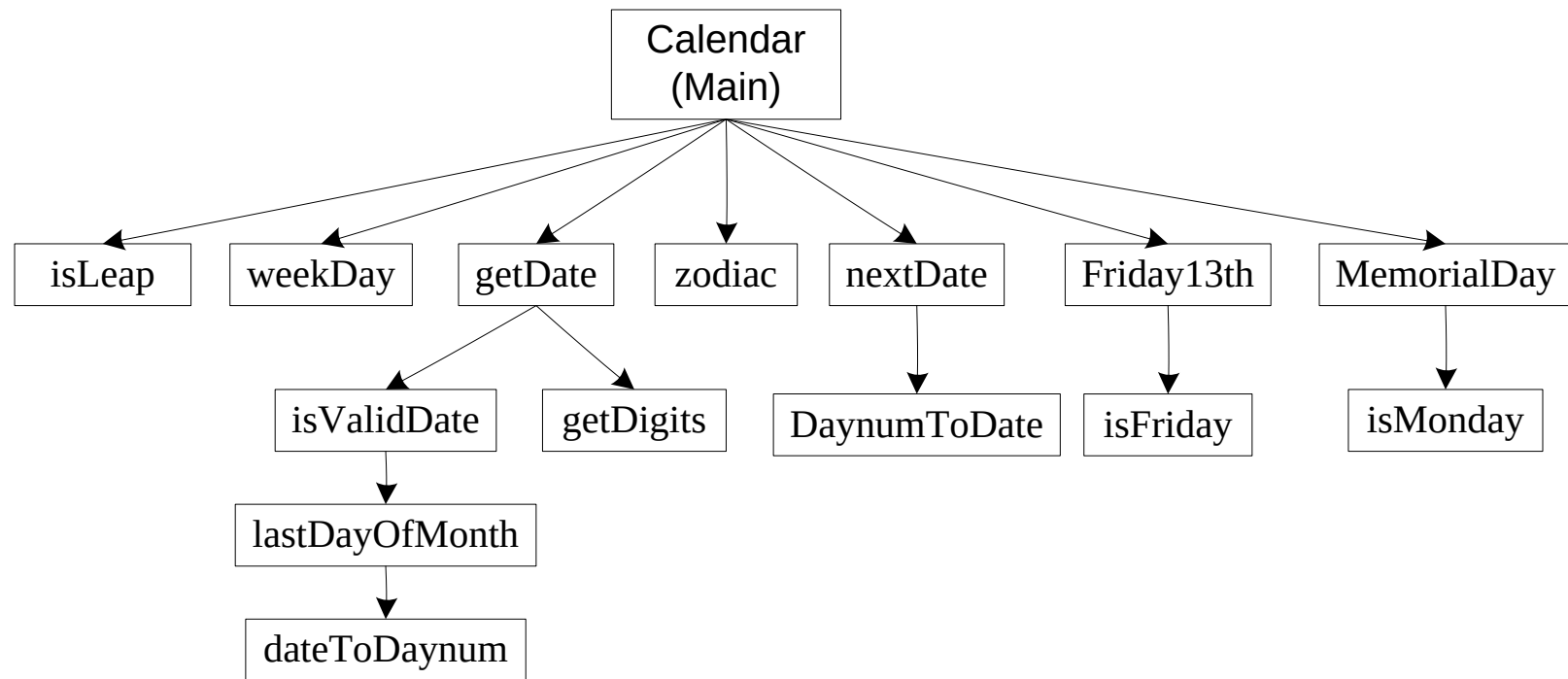


Procedural Calendar Program Units

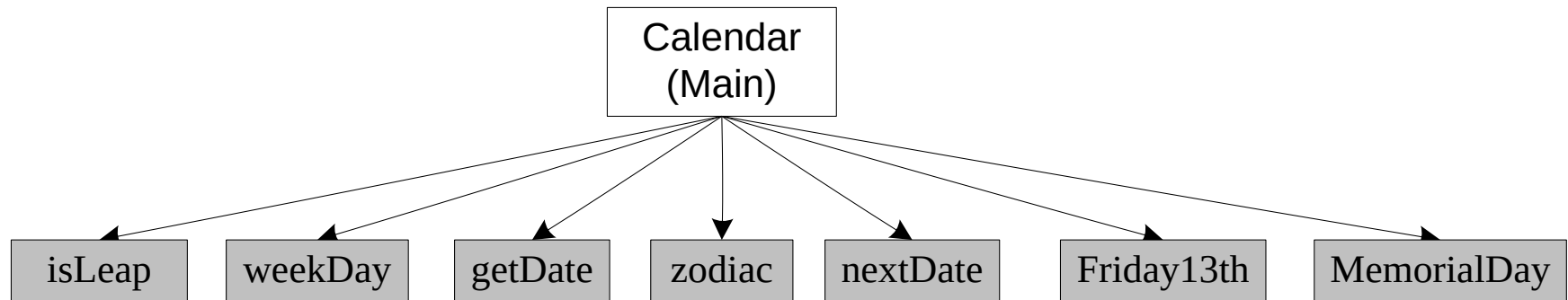
- Main
 - Calendar
 - Function isLeap
 - Procedure weekDay
 - Procedure getDate
 - Function isValidDate
 - Function lastDayOfMonth
 - Procedure getDigits
 - Procedure memorialDay
 - Function isMonday
 - Procedure friday13th
 - Function isFriday
 - Procedure nextDate
 - Procedure dayNumToDate
 - Procedure zodiac



Functional Decomposition of Calendar



First Step in Top-Down Integration



“Grey” units are stubs that return the correct values when referenced. This level checks the main program logic.



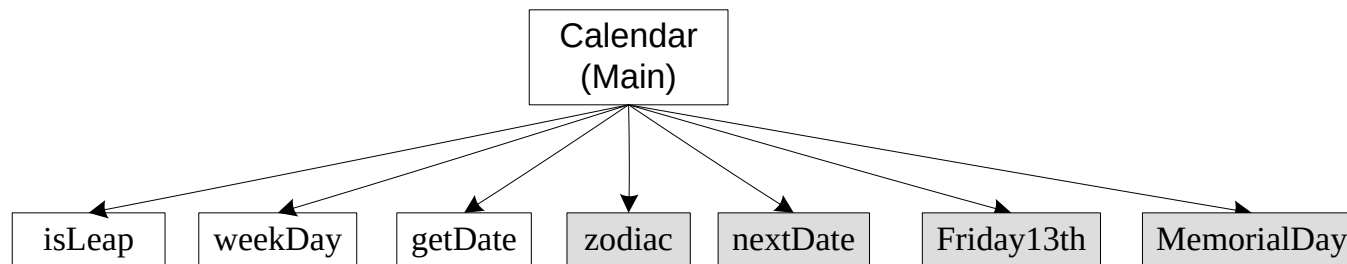
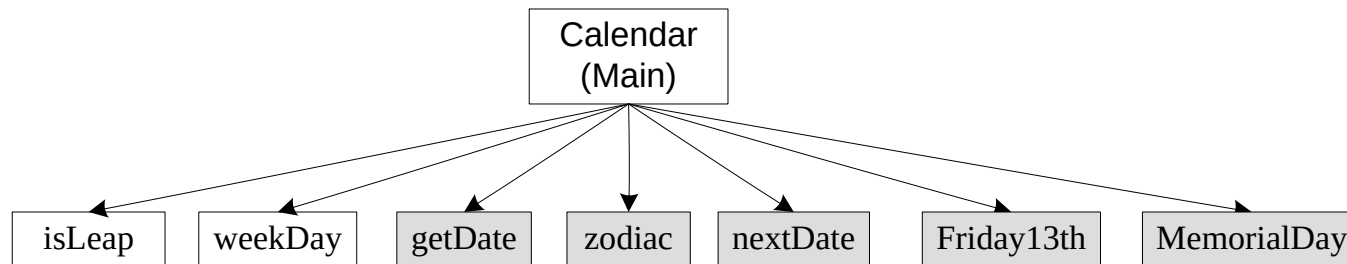
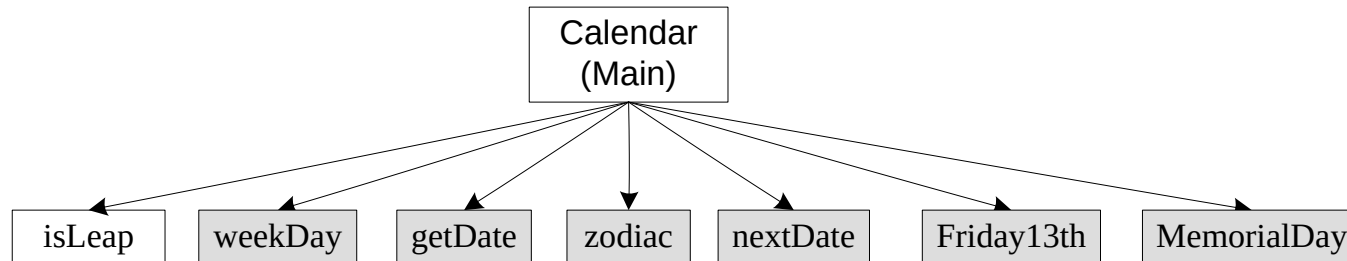
weekDayStub

```
Procedure weekDayStub(mm, dd, yyyy, dayName)
  If ((mm = 03) AND (dd = 25) AND (yyyy = 2021))
    Then dayName = "Thursday"
  EndIf
  .
  .
  .
  If ((mm = 03) AND (dd = 30) AND (yyyy = 2021))
    Then dayName = "Tuesday"
  EndIf
```



Next Three Steps

(replace one stub at a time with the actual code.)



Top-Down Integration Mechanism

- Breadth-first traversal of the functional decomposition tree.
- First step: Check main program logic, with all called units replaced by stubs that always return correct values.
- Move down one level
 - replace one stub at a time with actual code.
 - any fault must be in the newly integrated unit



Bottom-Up Integration Mechanism

- Reverse of top-down integration
- Start at leaves of the functional decomposition tree.
- Driver units...
 - call next level unit
 - serve as a small test bed
 - “drive” the unit with inputs
 - drivers know expected outputs
- As with top-down integration, one driver unit at a time is replaced with actual code.
- Any fault is (most likely) in the newly integrated code.

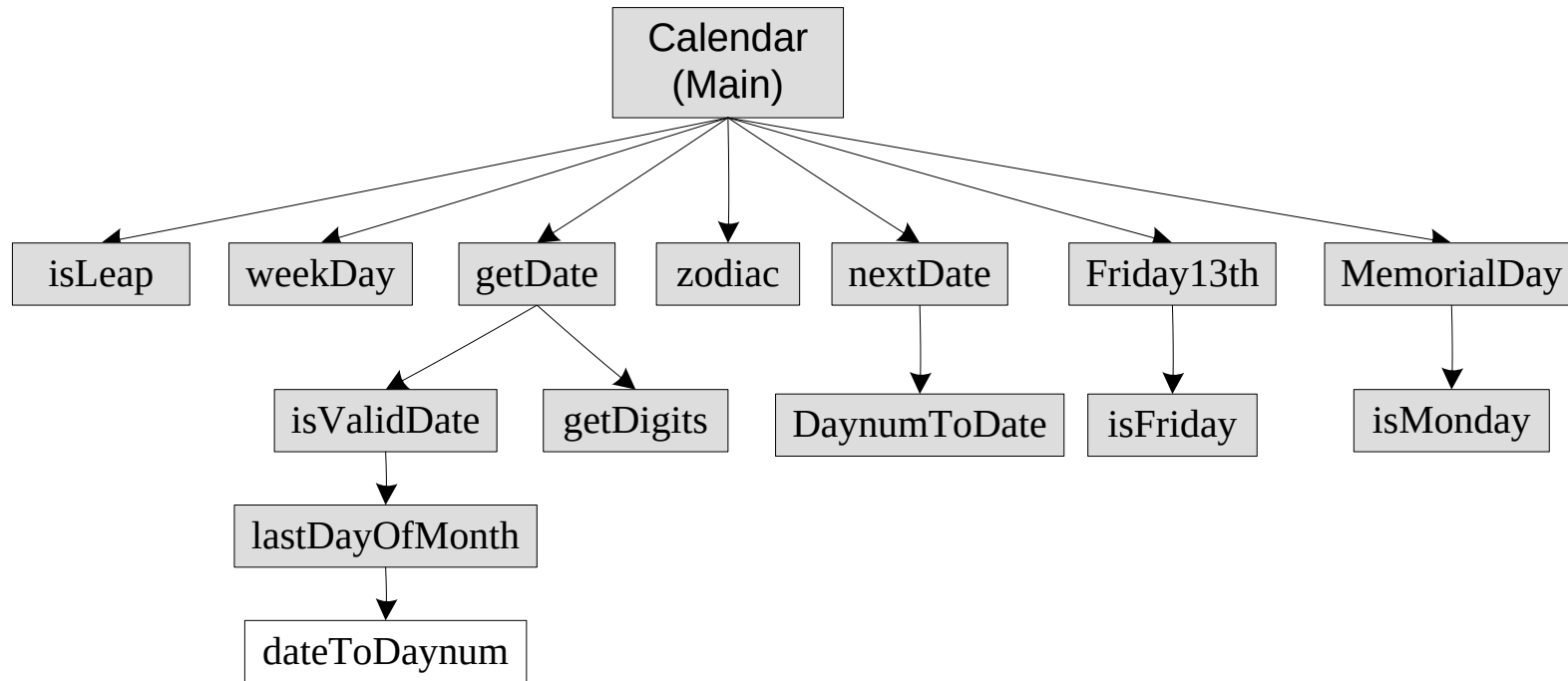


Top-Down and Bottom-Up Integration

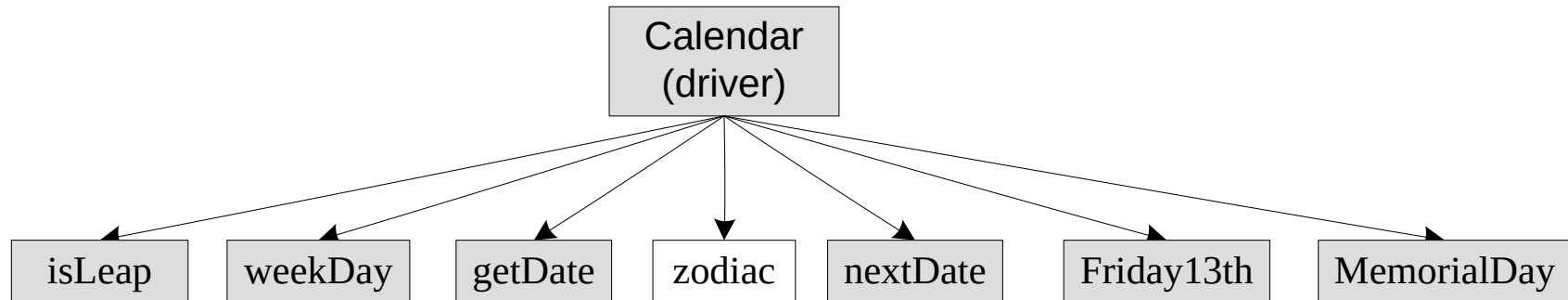
- Both depend on throwaway code.
 - drivers are usually more complex than stubs
- Both test just the interface between two units at a time.
- In Bottom-Up integration, a driver might simply reuse unit level tests for the “lower” unit.
- Fan-in and fan-out in the decomposition tree results in some redundancy.



Starting Point of Bottom-Up Integration



Bottom-Up Integration of Zodiac

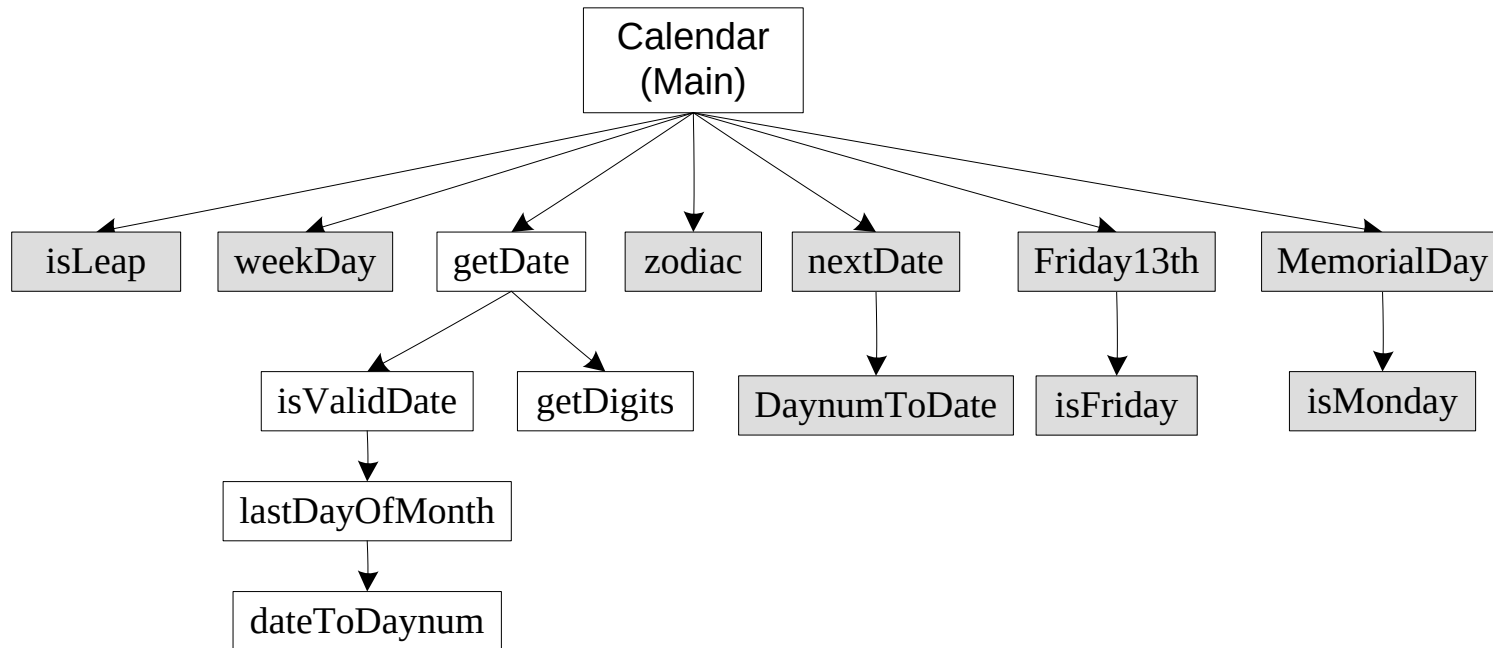


Sandwich Integration

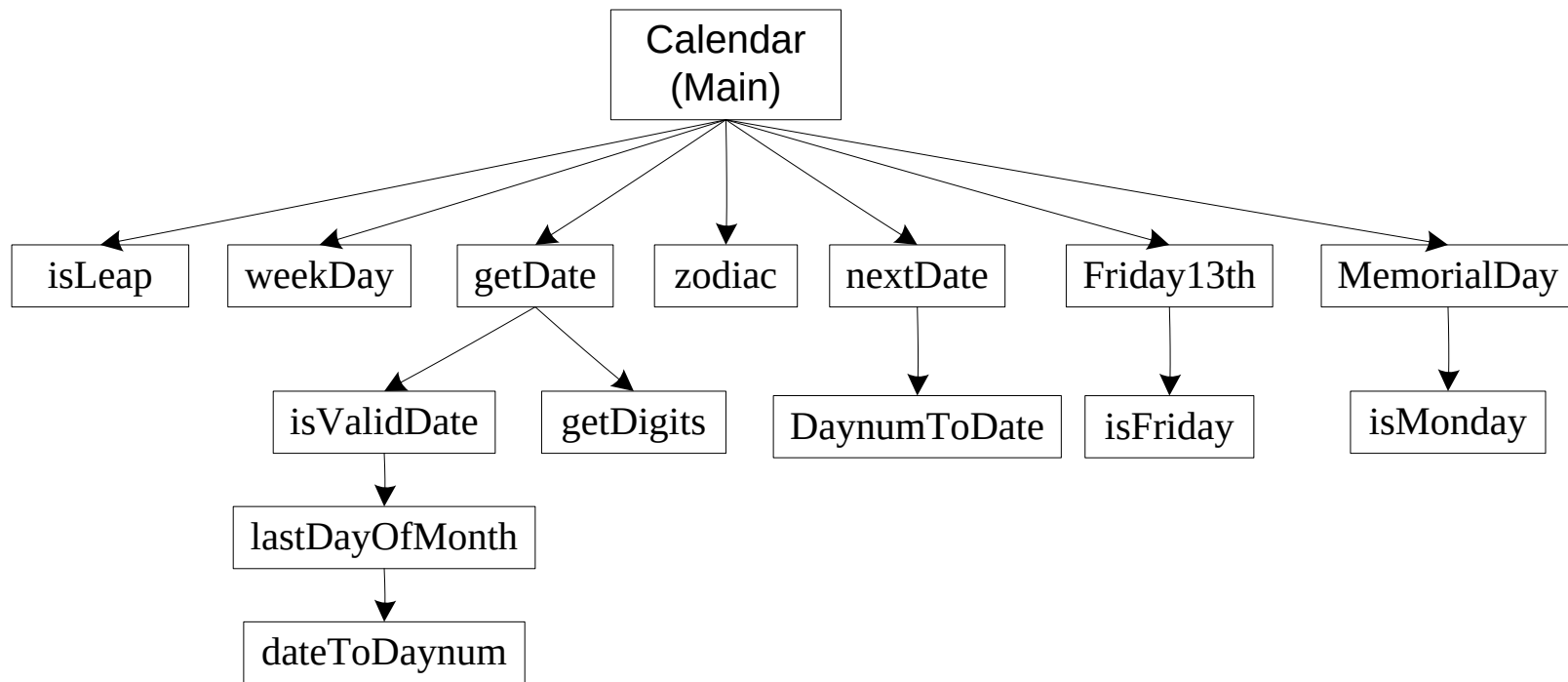
- Avoids some of the repetition on both top-down and bottom-up integration.
- Nicely understood as a depth-first traversal of the functional decomposition tree.
- A “sandwich” is one path from the root to a leaf of the functional decomposition tree.
- Avoids stub and driver development.
- More complex fault isolation.



A Sample Sandwich



“Big Bang” Integration



“Big Bang” Integration

- No...
 - stubs
 - drivers
 - strategy
- And very difficult fault isolation
- (Named after one of the theories of the origin of the Universe)
- This is the practice in an agile environment with a daily run of the project to that point.



Pros and Cons of Decomposition-Based Integration

- Pros
 - intuitively clear
 - “build” with proven components
 - fault isolation varies with the number of units being integrated
- Cons
 - based on lexicographic inclusion (a purely structural consideration)
 - some branches in a functional decomposition may not correspond with actual interfaces.
 - stub and driver development can be extensive

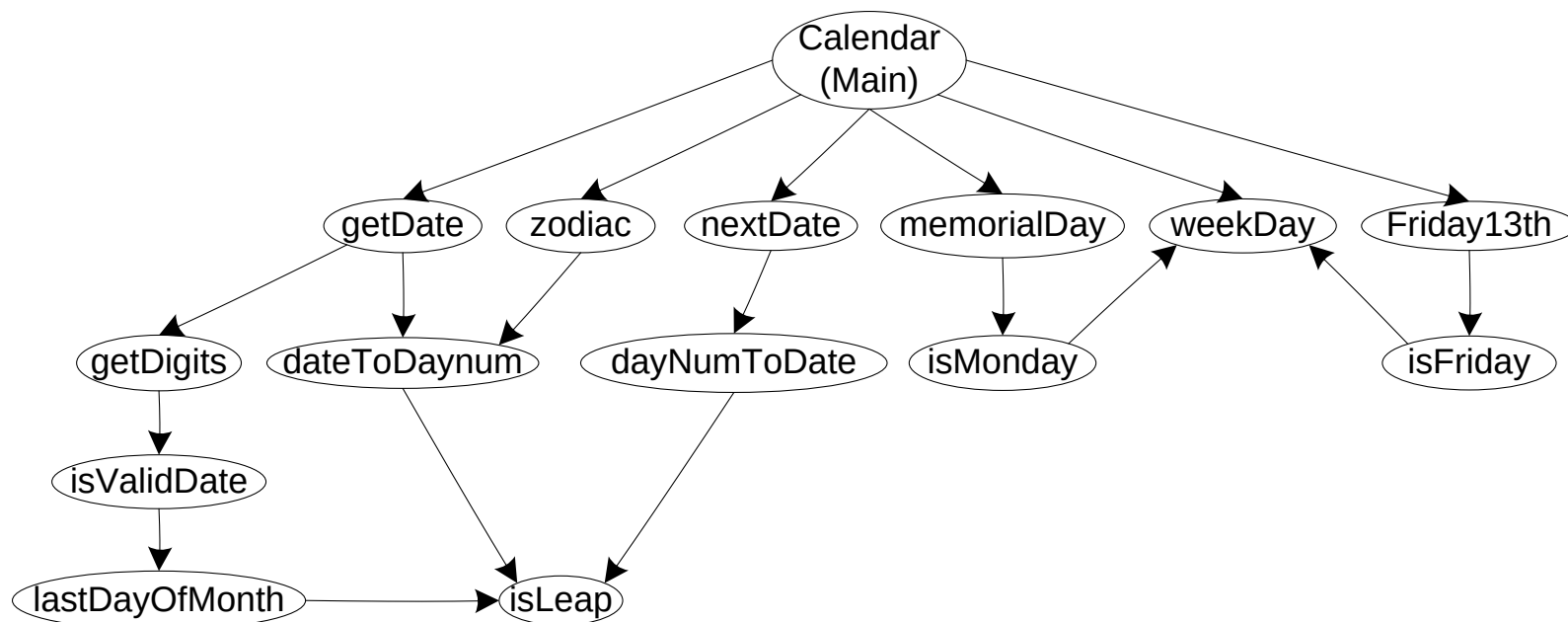


Call Graph-Based Integration

- Definition: The *Call Graph* of a program is a directed graph in which
 - nodes are unit
 - edges correspond to actual program calls (or messages)
- Call Graph Integration avoids the possibility of impossible edges in decomposition-based integration.
- Can still use the notions of stubs and drivers.
- Can still traverse the Call Graph in a top-down or bottom-up strategy.



Call Graph of the Calendar Program



Call Graph-Based Integration (continued)

- Two strategies
 - Pair-wise integration
 - Neighborhood integration
- Degrees of nodes in the Call Graph indicate integration sessions
 - isLeap and weekDay are each used by three units
- Possible strategies
 - test high indegree nodes first, or at least,
 - pay special attention to “popular” nodes

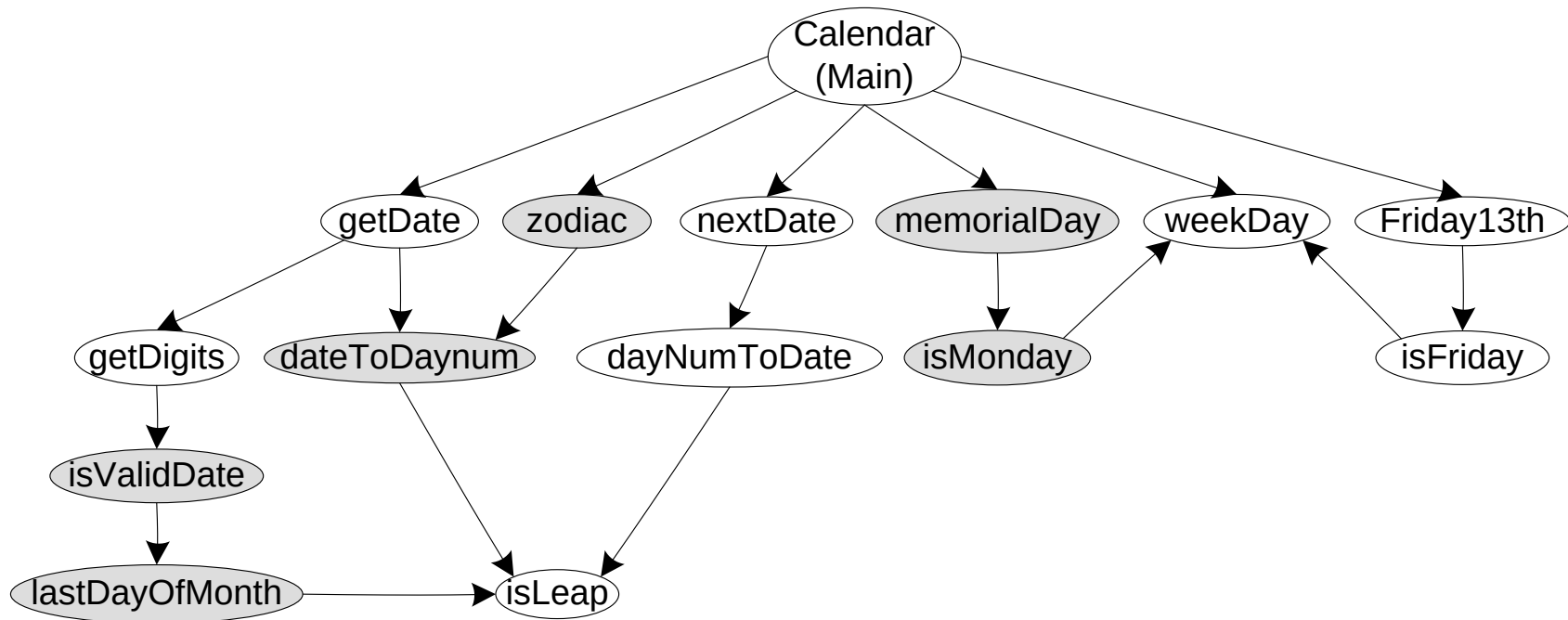


Pair-Wise Integration

- By definition, an edge in the Call Graph refers to an interface between the units that are the endpoints of the edge.
- Every edge represents a pair of units to test.
- Still might need stubs and drivers
- Fault isolation is localized to the pair being integrated



Nine Pairs for Pair-Wise Integration

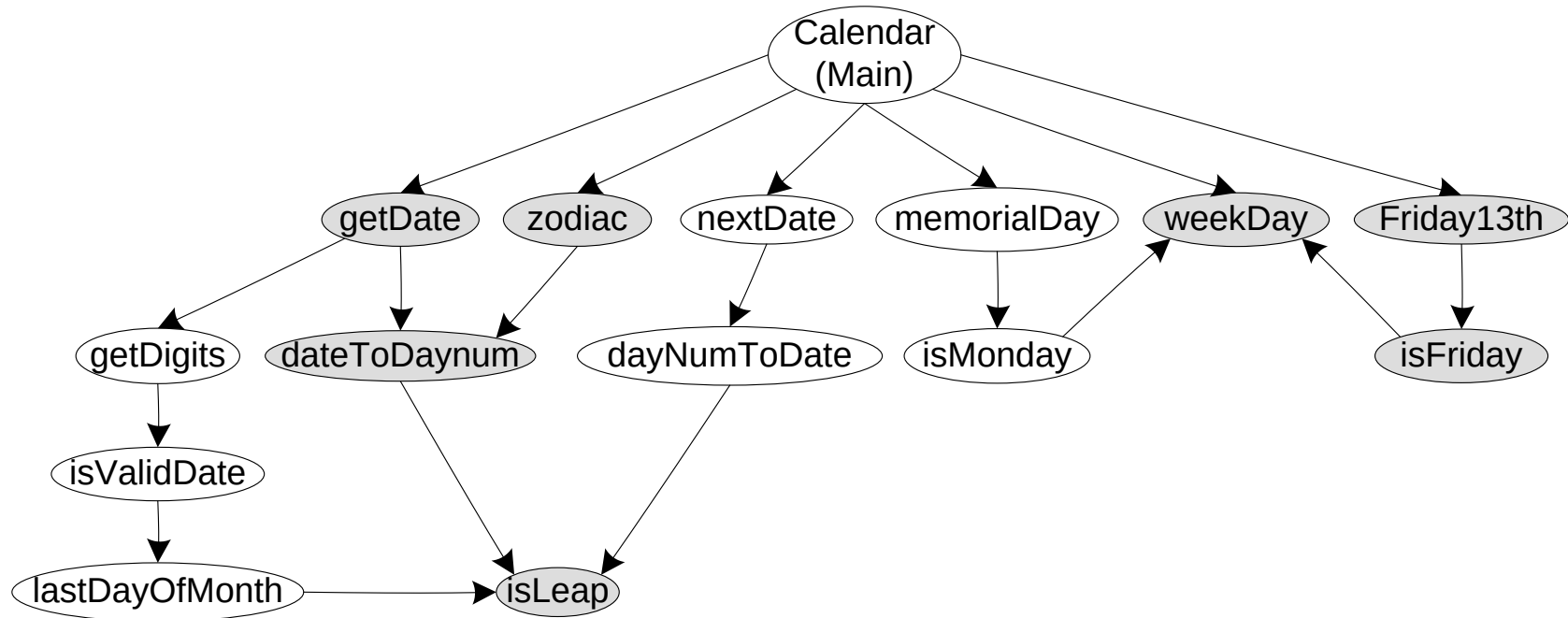


Neighborhood Integration

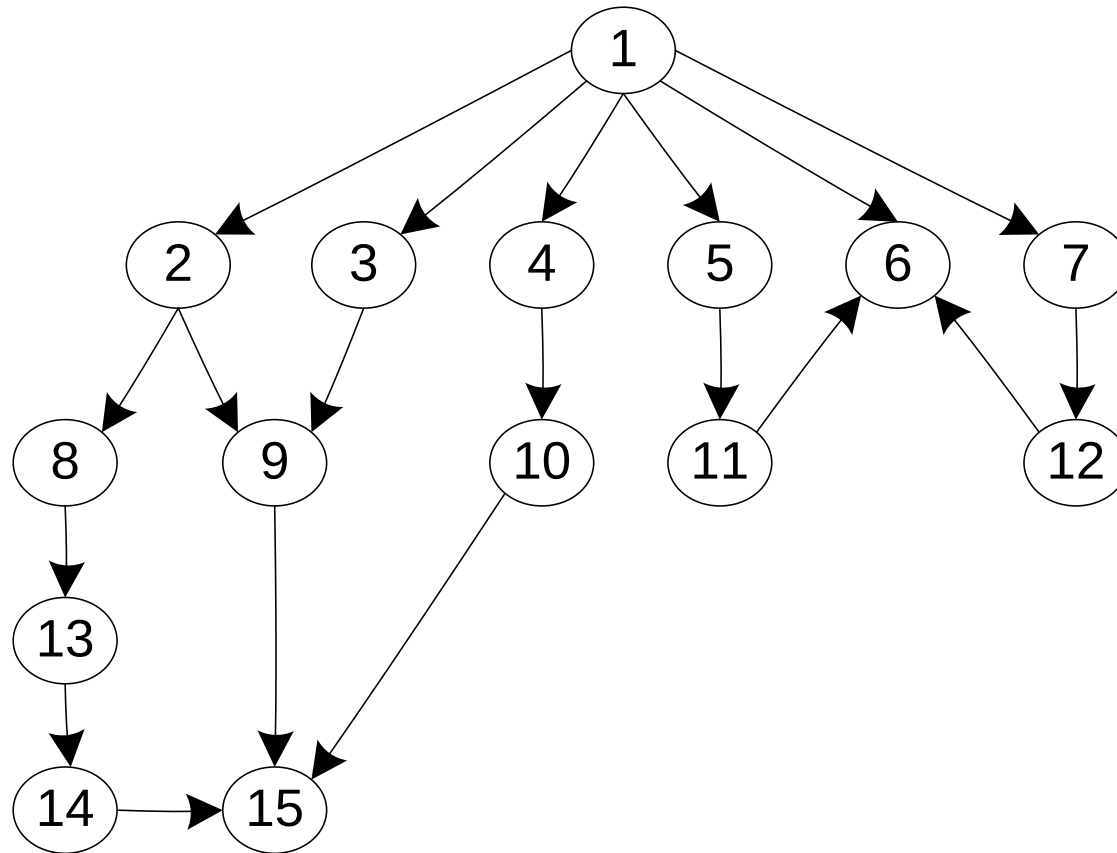
- The neighborhood (of radius 1) of a node in a graph is the set of nodes that are one edge away from the given node.
- This can be extended to larger sets by choosing larger values for the radius.
- Stub and driver effort is reduced.



Three Neighborhoods (radius = 1)



Calendar Call Graph with Numbered Nodes



Neighborhoods in the Calendar Program Call Graph			
Node	Unit name	Predecessors	Successors
1	Calendar (Main)	(none)	2, 3, 4, 5, 6, 7
2	getDate	1	8, 9
3	zodiac	1	9
4	nextDate	1	10
5	memorialDay	1	11
6	weekday	1, 11, 12	(none)
7	Friday13th	1	12
8	getDigits	2	13
9	dateToDayNum	3	15
10	dayNumToDate	4	15
11	isMonday	5	6
12	isFriday	7	6
13	isValidDate	8	14
14	lastDayOfMonth	13	15
15	isLeap	9, 10, 14	(none)



Path-Based Integration

- Wanted: an integration testing level construct similar to DD-Paths for unit testing...
 - extend the symbiosis of spec-based and code-based testing to the integration level
 - greater emphasis on behavioral threads
 - shift emphasis from interface testing to interactions (co-functions) among units
- Need some new definitions
 - source and sink nodes in a program graph
 - module (unit) execution path
 - generalized message
 - MM-Path



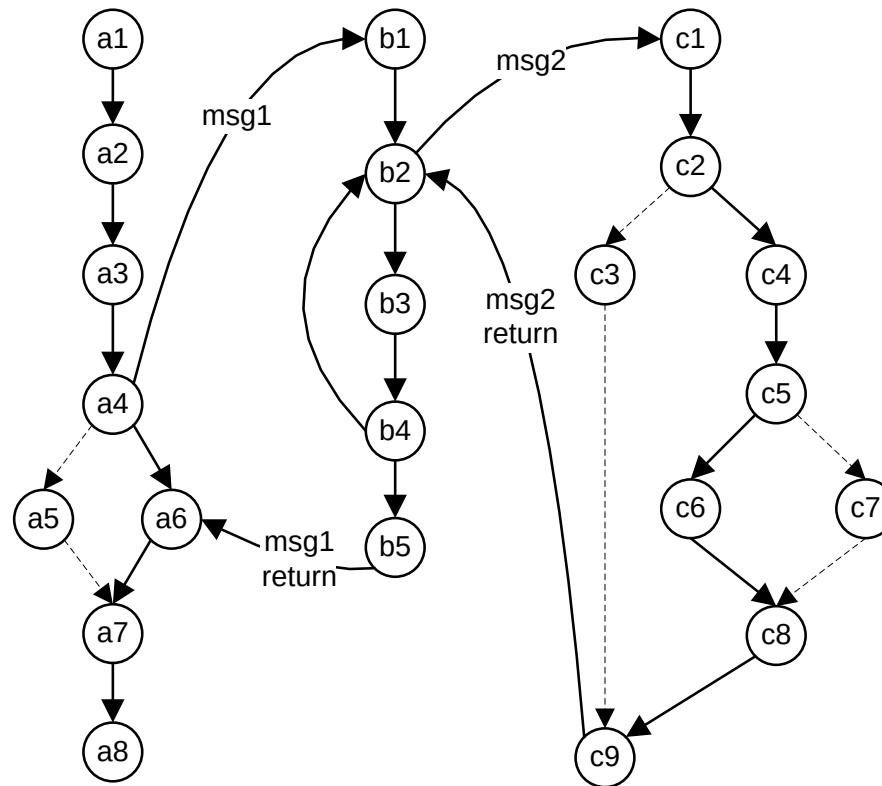
New and Extended Definitions

- A *source node* in a program is a statement fragment at which program execution begins or resumes.
- A *sink node* in a unit is a statement fragment at which program execution terminates.
- A *module execution path* is a sequence of statements that begins with a source node and ends with a sink node, with no intervening sink nodes.
- A *message* is a programming language mechanism by which one unit transfers control to another unit, and acquires a response from the other unit.



MM-Path Definition and Example

- An *MM-Path* is an interleaved sequence of module execution paths and messages.
- An MM-Path across three units:
- (message msg1 could return to node a4)



Exercise: List the source and sink nodes in the previous example.

Unit	Source Nodes	Sink Nodes
A		
B		
C		



Details of the Example MM-Path

The node sequence in the example MM-Path is:

<a1, a2, a3, a4>

message msg1

<b1, b2>

message msg2

<c1, c2, c4, c5, c6, c8, c9>

msg2 return

<b3, b4, (b2, b3, b4)*, b5>

msg1 return

<a6, a7, a8>

Note: the (b2, b3, b4)* is the Kleene Star notation for repeated traversal of the loop.



About MM-Paths

- Message quiescence: in a non-trivial MM-Path, there is always (at least one) a point at which no further messages are sent.
- In the example MM-Path, unit C is the point of message quiescence.
- In a data-driven program (such as NextDate), MM-Paths begin (and end) in the main program.
- In an event-driven program (such as the Garage Door Controller, MM-Paths begin with the unit that senses an input event and end in the method that produces the corresponding output event.



Some Sequences...

- A DD-Path is a sequence of source statements.
- A unit (module) execution path is a sequence of DD-Paths.
- An MM-Path is a sequence of unit (module) execution paths.
- A (system level) thread is a sequence of MM-Paths.
- Taken together, these sequences are a strong “unifying factor” across levels of testing. They can lead to the possibility of trade-offs among levels of testing (not explored here).



Cyclomatic Complexity for MM-Paths?

- The next two slides (figures 12.13 and 12.20) show potential MM-Paths in two examples.
- The solid edges in the second slide shows actual MM-Paths.
- Possibilities for their cyclomatic complexity is explored in the next few slides.
- Questions
 - cyclomatic complexity is ALWAYS understood as a static construct/quantity
 - BUT MM-Paths are dynamic by definition and design



Figure 12.13

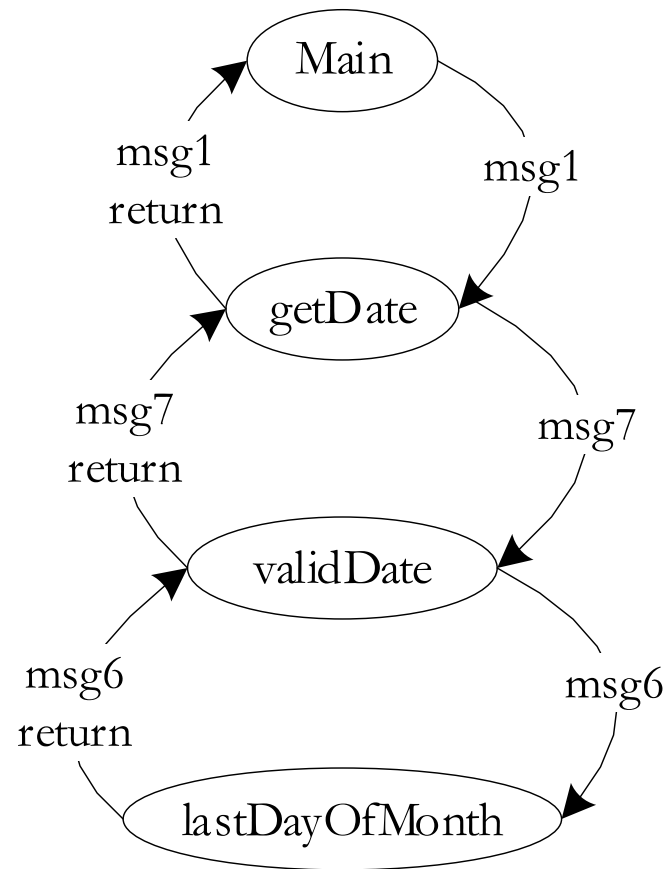
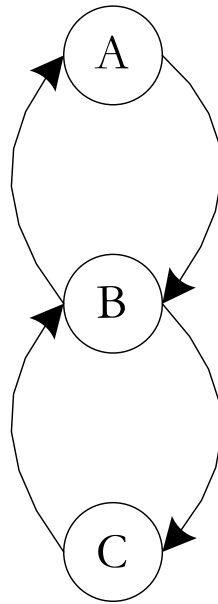
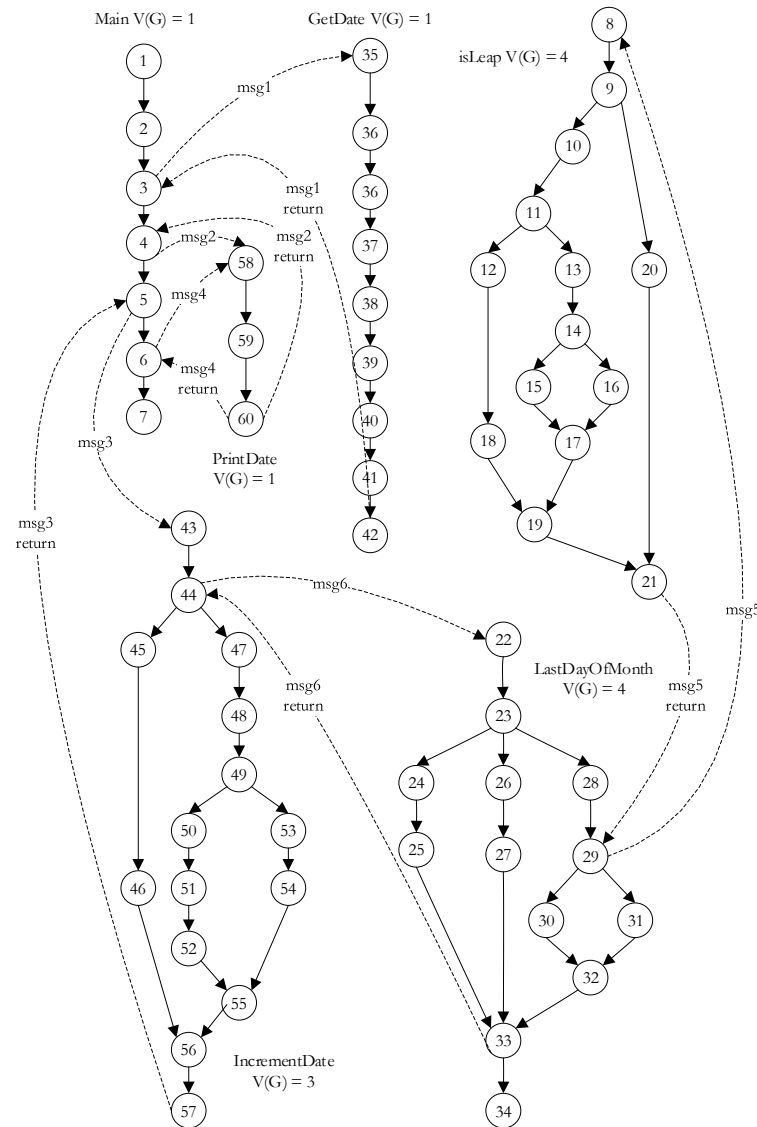


Figure 12.20



Cyclomatic Complexity for MM-Paths?

- Not a very good idea.
 - The message returns generally result small values of $V(G)$
- Consider the MM-Path that starts and ends at node 3 in the Main program.
 - $N = 9$
 - $E = 9$
 - $P = 1$
 - $V(G) = 1$
- Maybe the number of messages is a better indicator of complexity.



Complexity for MM-Paths?

- Consider the MM-Path that begins and ends at node 5 in the Main program.
 - Message sequence: $\langle \text{msg3}, \text{msg6}, \text{msg5} \rangle$
 - Message msg5 is the point of message quiescence
- Exercise: follow this MM-Path for Feb. 28, 2000
 - $N =$
 - $E =$
 - $P =$
 - $V(G) =$



Procedural Integration Strategies (Table 12.3)

Strategy Basis	Ability to test Interfaces	Ability to test co-functionality	fault isolation resolution
Functional Decomposition	acceptable but can be deceptive	limited to pairs of units	good, to faulty unit
Call Graph	acceptable	limited to pairs of units	good, to faulty unit
MM-Path	excellent	complete	excellent, to faulty unit execution path



Object-Oriented Calendar Program Elements

- Classes
 - DateTest, 1 method, $V(G) = 1$
 - Date, 2 methods, $V(G) = 3$
 - Day, 2 methods, $V(G) = 5$
 - Month, 3 methods, $V(G) = 10$
 - Year, 3 methods, $V(G) = 6$
- 24 messages (see Table 12.5)
- Message flows
 - See Figure 12.22 for all message flows
 - See Figure 12.23 for message flows for May 27, 2020



Comparison of Procedural and O-O Implementations (Table 12.4)

	Procedural	Object-oriented
Number of units	6	5 classes, 25 methods
Sum of unit complexities	14	40
Number of lines of code	90	119
Number of messages	7	24

Some Conclusions

- O-O shifts complexity from unit to integration level
- MM-Paths work well for procedural code
- MM-Paths are more complex for o-o code
- Model-Based Integration Testing? (explored next)

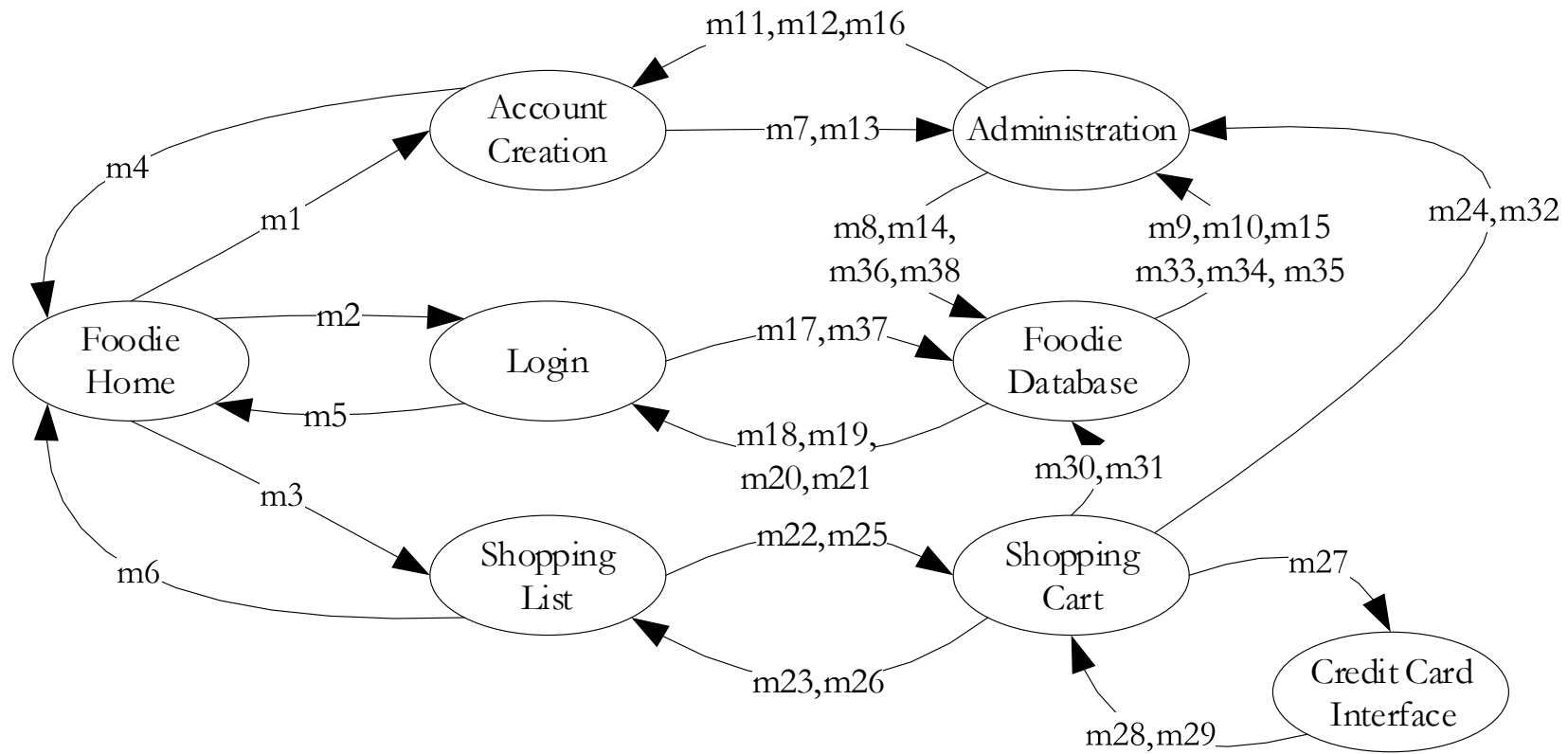


Model-Based Integration Testing

- O-O shifts complexity from unit to integration level
- Increased complexity suggests a higher level model
- Message-based communication among communicating finite state machines
- Use many ideas from Call Graph based integration
- Example: the Foodies Wish List



Model of the Foodie Wish List



Message Communication in the Foodie Wish List

- Simplest form of Model-Based Integration
 - Verify every message sent to correct recipient
 - Verify every message received by correct recipient
- Use ASSERT statements in a JUnit-like environment



Pairwise Integration in the Foodie Wish List

- Eight pairs of communicating FSMs
- Foodie Home with
 - Account Creation
 - Login
 - Shopping List
- Account Creation with Administration
- Login with Foodie Database
- Shopping List with Shopping Cart
- Administration with Foodie Database
- Shopping Cart with
 - Administration
 - Credit Card Interface



Closer Look at Account Creation and Administration

- Communicate with 5 messages
- Just testing the message communication is an oversimplification
 - Very little semantic content
 - Response messages can be context dependent
 - *E.g.*, replies to message m7 could be either m11 or m12 depending on Whether m7 introduces a new UserID
- Moving in the direction of WHY a message is sent
- Shopping Cart with
 - Administration
 - Credit Card Interface



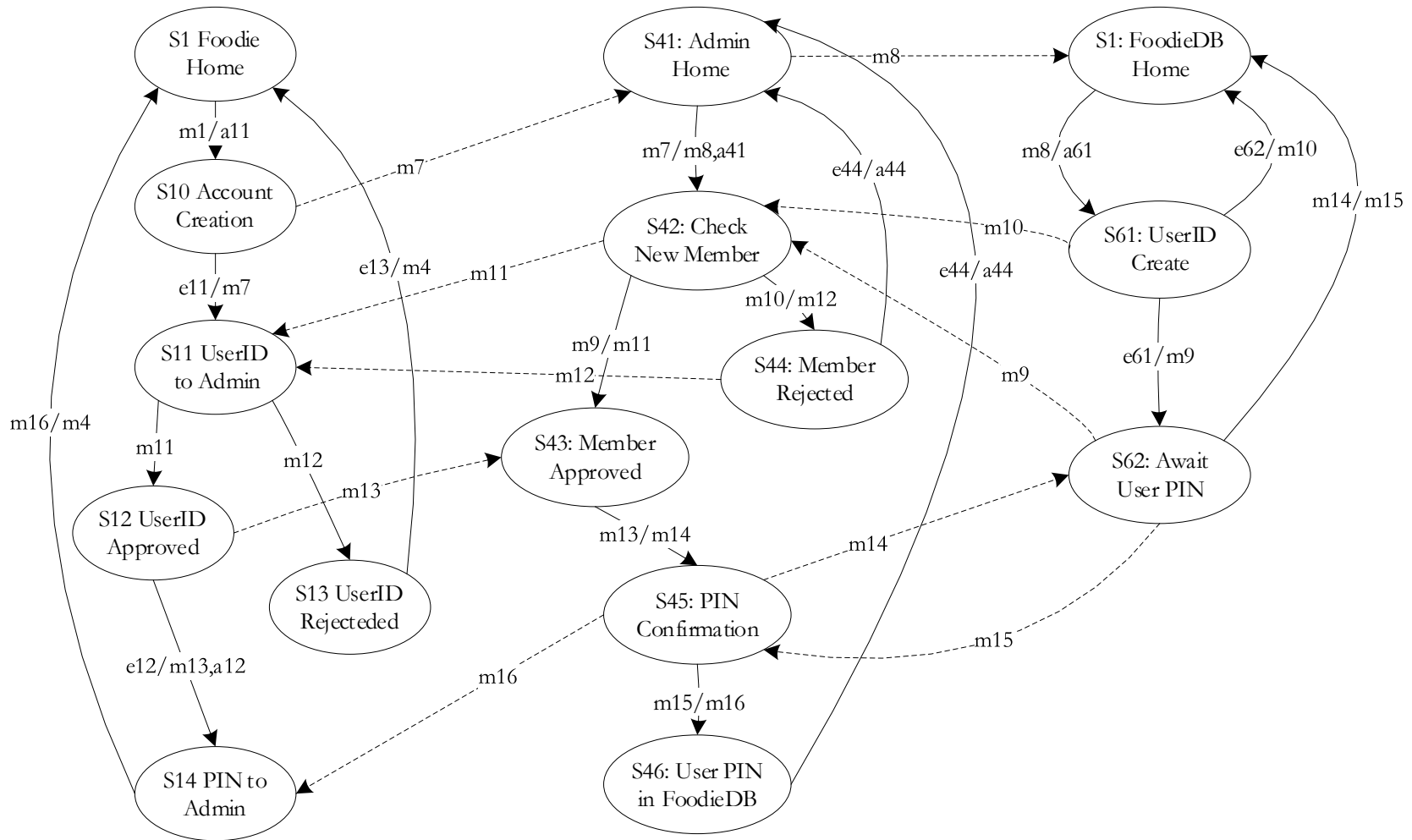
Pairwise Test Case 1 (3 pairs)

	Original UserID Entered	
Test Case ID	pw1	
Description	User enters an original UserID	
Preconditions	The entered USerID is not in FoodieDB	
Message Sequence		
Source	Message or Input	Destination
AccountCreationDriver	m1	Account Creation
Account Creation	m7	Admin
Admin	m8	FoodieDBStub
FoodieDBStub	m9	Admin
Admin	m11	Account Creation



Three Communicating Foodie Wish List FSMs

(figure 12.25)



Pairwise Test Case 2 (3 pairs)

Test Case Name	Original UserID and PIN entry	
Test Case ID	pw2	
Description	User enters an original UserID and a PIN	
Preconditions	Entered UserID is not in FoodieDB, PIN is OK	
Message Sequence		
Source	Message or Input	Destination
Account Creation	m7	Admin
Admin	m8	FoodieDBStub
FoodieDBStub	m9	Admin
Admin	m11	Account Creation
Account Creation	m13	Admin
Admin	m14	FoodieDBStub
FoodieDBStub	m15	Admin
Admin	m16	Account Creation



Pairwise Test Case 3 (3 pairs)

Test Case Name	Duplicate UserID Entered	
Test Case ID	pw3	
Description	User enters a duplicate UserID	
Preconditions	The entered USerID is in FoodieDB	
Message Sequence		
Source	Message or Input	Destination
Account Creation	m7	Admin
Admin	m8	FoodieDBStub
FoodieDBStub	m10	Admin
Admin	m12	Account Creation



FSM/M Path Integration

Definition

A Finite State Machine Message Path (FSM/M) is a path that originates and terminates with a Finite State Machine and connects to other FSMs using messages

Notation

An FSM/M Path is denoted by the (interior) sequence of messages. The FSM/M Path in Pairwise Test Case 1 is the message sequence $\langle m7, m8, m9, m11 \rangle$.



Hypothetical Test Tool Language

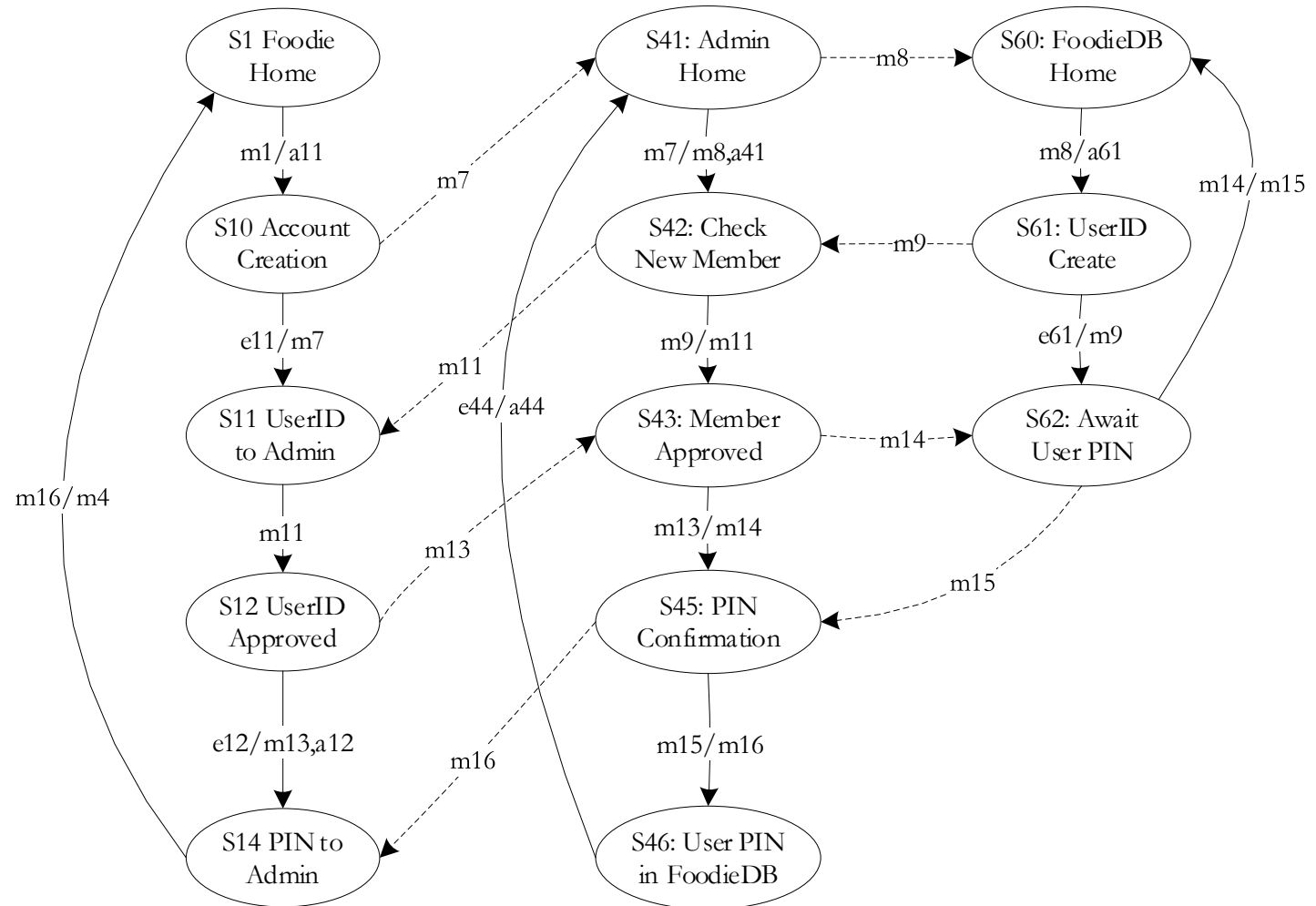
- Consider an integration testing tool that allows a tester to “execute” paths through a set of communicating finite state machines
- The verbs CAUSE and VERIFY are used as follows.
- A CAUSE statement can do any of the following:
 - cause an event local to a fsm (e.g., e11),
 - cause message to be sent to an adjacent fsm, with parameters as needed,
 - cause a state transition.
- VERIFY statements can:
 - recognize the present state of the fsm in which it occurs,
 - report the name of the present state,
 - verify the value of VERIFY parameters



FSM/M Scenario 1

Step	In State	CAUSE Event/ Message	NextState	Result (VERIFY)
1	S1	m1	S10	in AccountCreation S10
2	S10	e11	S11	in AccountCreation S11
3	S10	m7: propose UserID = 'Paul'	S41	in Admin; UserID = 'Paul'
4	S41	m7	S42	in Admin;
5	S41	m8	S60	in FoodieDB; UserID = 'Paul'
6	S60	m8	S61	in FoodieDB;
7	S61	e61	S62	in FoodieDB;
8	S61	m9: Approve UserID = 'Paul'	S42	in Admin, UserID = 'Paul' OK
9	S42	m9	S43	in Admin
10	S42	m11	S11	in AccountCreation
11	S11	m11	S12	UserID = 'Paul' approved
12	S12	e12	S14	in AccountCreation
13	S12	m13: UserPIN defined	S43	in Admin: pass on UserPIN
14	S43	m14: UserPIN to FoodieDB	S62	UserPIN stored in FoodieDB
15	S62	m15: Confirm UserPIN	S45	in Admin
16	S45	m15	S46	in Admin
17	S45	m16: Defined PIN accepted	S14	in AccountCreation
18	S46	e44	S41	in Admin Home
19	S14	m16: Defined PIN accepted	S1	in Foodie Home

FSM/M Scenario 1 (figure 12.26)



Test Cases from FSM/M Scenarios

- Can be derived directly (automatically?)
- Excellent for requirements tracing
- Easy to integrate with a test tool
- BUT tests are only as good/complete as the underlying model
- Bottom line? Worth the extra effort



















